

SOB4ES - Comparación de Modelos

CRISP-ML(Q) - Fase 5: Evaluación y selección del modelo final

Este notebook carga los modelos entrenados en los notebooks anteriores y compara su rendimiento sobre `eval.csv` de forma sistemática.

Modelos comparados:

1. Random Forest individual (modelos por target)
2. XGBoost individual (modelos por target)
3. Ridge Regression individual (modelos por target)
4. Random Forest multisalida
5. XGBoost multisalida
6. Ensemble de RegressorChains
7. MLP multisalida estándar
8. MLP con pérdida personalizada de correlación

Métricas de comparación:

- R2 global (promedio sobre todos los targets)
- R2 por target (para identificar grupos biológicos difíciles de predecir)
- RMSE y MAE globales

1.- Configuración

Configuración inicial del entorno, con librerías, variables globales y todo cargado.

```
In [1]: import os
import warnings
import joblib
import pandas as pd
import numpy as np
import itertools

import matplotlib.pyplot as plt

import torch
import torch.nn as nn

from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
from sklearn.metrics import precision_score, recall_score, f1_score

warnings.filterwarnings('ignore')

DATA_DIR = 'input/'
MODELS_BASE = 'output/models/'

DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

TARGETS = [
    # Shannon diversity index
    'nematode_shannon_z',
    'macro_shannon_z',
    'earthworm_shannon_z',
    'orib_shannon_z',
    'meso_shannon_z',
    'coll_shannon_z',
    'bac_shannon_z',
    'fun_shannon_z',
    'euk_shannon_z',
    'oomy_shannon_z',
    'cerc_shannon_z',
    # Richness
    'macro_order_richness_z',
    'earthworm_richness_z',
    'orib_species_richness_z',
    'meso_species_richness_z',
    'coll_species_richness_z',
    'bac_asv_richness_z',
    'fun_asv_richness_z',
    'euk_asv_richness_z',
    'oomy_asv_richness_z',
    'cerc_asv_richness_z'
]

FEATURES_AUTORIZADAS = [
    'total_plant_cover_z',
    'clay_content_z',
    'silt_content_z',
    'sand_content_z',
    'aggregate_stability_z',
    'bulk_density_z',
    'soil_moisture_z',
    'as_z',
    'cu_z',
    'k_z',
    'mo_z',
    'ni_z',
    'p_z',
    'pb_z',
```

```

'zn_z',
'soil_ph_z',
'plot_total_organic_c_z',
'plot_total_n_z',
'gee_temp_media_C_z',
'gee_humedad_rel_pct_z',
'gee_ndvi_verano_z',
'dem_elevacion_m_z',
'dem_pendiente_deg_z',
'dem_orientacion_deg_z',
'eu_clay_content_z',
'eu_sand_content_z',
'eu_silt_content_z',
'eu_water_holding_capacity_z',
'eu_cn_ratio_z',
'eu_p_z',
'eu_ph_z',
'eu_as_z',
'eu_organic_carbon_octop_z',
'eu_zn_z'
]

print('Configuracion cargada.')

# Paleta de colores unificada
PALETTE = {
    'blue':      '#7FA6C9',
    'blue_light': '#AAD0F1',
    'green':     '#7FC8A9',
    'green_light': '#BEE6D3',
    'purple':    '#A98FC7',
    'purple_light': '#D6C7EA',
    'gray':      '#9B9B9B',
    'gray_dark': '#5C5C5C',
}

# Secuencia ciclica para graficos con N series/categorias (N puede superar 6)
PALETTE_CYCLE = [PALETTE['blue'], PALETTE['green'], PALETTE['purple'],
                  PALETTE['blue_light'], PALETTE['green_light'], PALETTE['purple_light']]

def palette_cycle(n):
    """Devuelve una lista de n colores repitiendo PALETTE_CYCLE si hace falta."""
    return list(itertools.islice(itertools.cycle(PALETTE_CYCLE), n))

```

Configuracion cargada.

2.- Carga de eval.csv

Se carga únicamente el dataset de evaluación final. Ninguno de los modelos ha visto estos datos durante el entrenamiento ni el tuning.

```

In [2]: df_eval = pd.read_csv(os.path.join(DATA_DIR, 'eval.csv'))
X_eval = df_eval[FEATURES_AUTORIZADAS]
y_eval = df_eval[TARGETS]

print(f'eval.csv cargado: {X_eval.shape[0]} muestras x {X_eval.shape[1]} variables')

```

eval.csv cargado: 65 muestras x 34 variables

3.- Funciones auxiliares

Se definen las funciones necesarias para cargar cada tipo de modelo y calcular sus predicciones sobre `eval.csv`.

```

In [3]: def metricas(y_true, y_pred, nombre):
    """Calcula R2, RMSE y MAE globales y devuelve un dict con los resultados."""
    r2 = r2_score(y_true, y_pred, multioutput='uniform_average')
    rmse = np.sqrt(mean_squared_error(y_true, y_pred, multioutput='uniform_average'))
    mae = mean_absolute_error(y_true, y_pred, multioutput='uniform_average')
    r2_targets = r2_score(y_true, y_pred, multioutput='raw_values')
    return {'modelo': nombre, 'R2': round(r2, 4), 'RMSE': round(rmse, 4), 'MAE': round(mae, 4),
            'r2_targets': r2_targets}

def predecir_ensamble_pkl(carpeta, prefijo, sufijo, n, X):
    """Carga n modelos .pkl y devuelve el promedio de sus predicciones."""
    preds = []
    for i in range(n):
        m = joblib.load(os.path.join(MODELS_BASE, carpeta, f'{prefijo}-{i}-{sufijo}.pkl'))
        preds.append(m.predict(X))
    return np.mean(preds, axis=0)

def predecir_ensamble_por_target(carpeta, prefijo_tpl, n, X):
    """
    Para modelos individuales (un modelo por target):
    carga n modelos por target y devuelve la matriz de predicciones.
    prefijo_tpl debe ser una funcion que recibe (target, seed_id) y devuelve el nombre del archivo.
    """
    y_pred = np.zeros((len(X), len(TARGETS)))
    for j, target in enumerate(TARGETS):
        preds_t = []
        for i in range(n):
            ruta = os.path.join(MODELS_BASE, carpeta, prefijo_tpl(target, i))
            m = joblib.load(ruta)
            preds_t.append(m.predict(X))
        y_pred[:, j] = np.mean(preds_t, axis=0)
    return y_pred

```

```

class MLPMultiSalida(nn.Module):
    def __init__(self, n_inputs, n_outputs, hidden_sizes, dropout_rate):
        super().__init__()
        layers, in_size = [], n_inputs
        for h in hidden_sizes:
            layers += [nn.Linear(in_size, h), nn.ReLU(), nn.Dropout(dropout_rate)]
            in_size = h
        layers.append(nn.Linear(in_size, n_outputs))
        self.red = nn.Sequential(*layers)
    def forward(self, x): return self.red(x)

def predecir_ensamble_mlp(carpeta, prefijo, n, X, config_file):
    """Carga n redes PyTorch y devuelve el promedio de sus predicciones."""
    cfg = joblib.load(os.path.join(MODELS_BASE, carpeta, config_file))
    X_t = torch.tensor(X.values, dtype=torch.float32).to(DEVICE)
    preds = []
    for i in range(n):
        model = MLPMultiSalida(X.shape[1], len(TARGETS), cfg['hidden'], cfg['dropout']).to(DEVICE)
        model.load_state_dict(torch.load(
            os.path.join(MODELS_BASE, carpeta, f'{prefijo}-{i}.pt'), map_location=DEVICE
        ))
        model.eval()
        with torch.no_grad():
            preds.append(model(X_t).cpu().numpy())
    return np.mean(preds, axis=0)

print('Funciones auxiliares definidas.')

```

Funciones auxiliares definidas.

4.- Predicciones de cada modelo

Se cargan y ejecutan todos los modelos sobre `eval.csv`. Este proceso puede tardar varios minutos dependiendo del número de modelos y el hardware disponible.

```

In [4]: resultados = []
y_true = y_eval.values

print('Cargando modelos y generando predicciones...\n')

# 1. Random Forest individual
print(' [1/8] Random Forest individual...')
y_pred = predecir_ensamble_por_target('rf', lambda t, i: f'rf_prod_{t}_m{i}.pkl', 5, X_eval)
resultados.append(metricas(y_true, y_pred, 'RF Individual'))

# 2. XGBoost individual
print(' [2/8] XGBoost individual...')
y_pred = predecir_ensamble_por_target('xgb', lambda t, i: f'xgb_prod_{t}_exp{i}.pkl', 5, X_eval)
resultados.append(metricas(y_true, y_pred, 'XGBoost Individual'))

# 3. Ridge individual
print(' [3/8] Ridge Regression...')
y_pred_ridge = np.zeros((len(X_eval), len(TARGETS)))
for j, target in enumerate(TARGETS):
    m = joblib.load(os.path.join(MODELS_BASE, 'reg', f'ridge_prod_{target}.pkl'))
    y_pred_ridge[:, j] = m.predict(X_eval)
resultados.append(metricas(y_true, y_pred_ridge, 'Regresión Individual'))

# 4. RF multisalida
print(' [4/8] Random Forest multisalida...')
y_pred = predecir_ensamble_pkl('rf_multi', 'rf_multi_m', '', 5, X_eval)
resultados.append(metricas(y_true, y_pred, 'RandomForest Multisalida'))

# 5. XGBoost multisalida
print(' [5/8] XGBoost multisalida...')
y_pred = predecir_ensamble_pkl('xgb_multi', 'xgb_multi_exp', '', 5, X_eval)
resultados.append(metricas(y_true, y_pred, 'XGBoost Multisalida'))

# 6. RegressorChain
print(' [6/8] Ensemble RegressorChain...')
N_CHAINS = 10
preds_chain = []
for chain_id in range(N_CHAINS):
    cadena = joblib.load(os.path.join(MODELS_BASE, 'chain', f'chain_{chain_id}.pkl'))
    preds_chain.append(cadena.predict(X_eval))
y_pred = np.mean(preds_chain, axis=0)
resultados.append(metricas(y_true, y_pred, 'Random Forest con RegressorChain'))

# 7. MLP estandar
print(' [7/8] MLP multisalida estandar...')
y_pred = predecir_ensamble_mlp('mlp', 'mlp_prod', 5, X_eval, 'mlp_config.pkl')
resultados.append(metricas(y_true, y_pred, 'MLP Estandar'))

# 8. MLP con perdida personalizada
print(' [8/8] MLP perdida personalizada...')
y_pred = predecir_ensamble_mlp('mlp_custom', 'mlp_custom', 5, X_eval, 'mlp_custom_config.pkl')
resultados.append(metricas(y_true, y_pred, 'MLP Perdida Custom'))

print('\nPredicciones completadas...')

```

Cargando modelos y generando predicciones...

```

[1/8] Random Forest individual...
[2/8] XGBoost individual...
[3/8] Ridge Regression...
[4/8] Random Forest multisalida...
[5/8] XGBoost multisalida...

```

```
[6/8] Ensemble RegressorChain...
[7/8] MLP multisalida estandar...
[8/8] MLP perdida personalizada...
```

Predicciones completadas...

5.- Tabla comparativa global

Se muestra la comparación ordenada por R2 descendente.

El mejor modelo es el que tiene R2 más alto y RMSE/MAE más bajos.

```
In [5]: tabla = pd.DataFrame([{'Modelo': r['modelo'],
'R2 global': r['R2'],
'RMSE': r['RMSE'],
'MAE': r['MAE']} for r in resultados]).sort_values('R2 global', ascending=False).reset_index(drop=True)

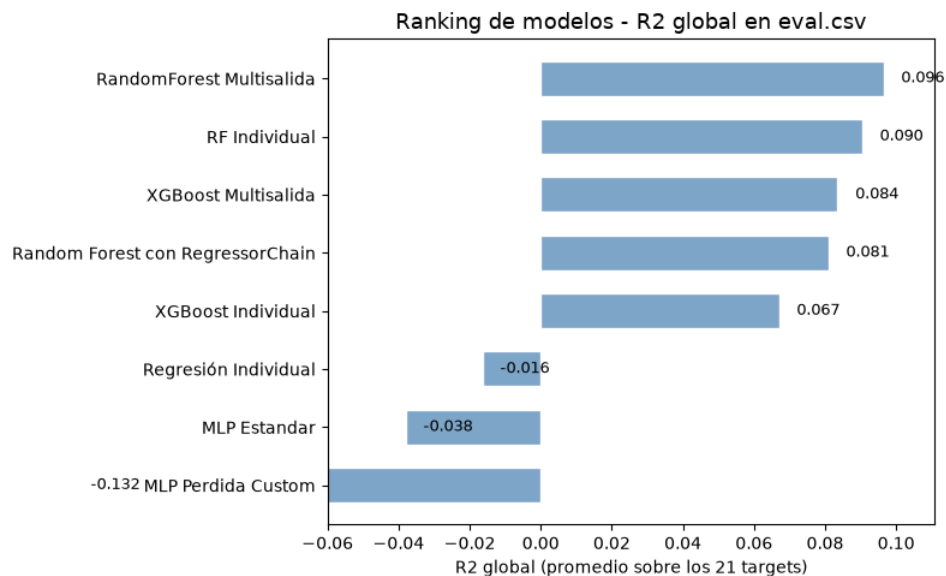
tabla.index += 1 # ranking desde 1
print('Comparacion de modelos (ordenado por R2 descendente):')
print('=' * 60)
print(tabla.to_string())
print('=' * 60)
print(f'\nMejor modelo: {tabla.iloc[0]["Modelo"]} (R2 = {tabla.iloc[0]["R2 global"]})')

# Grafico: ranking de modelos por R2 global
fig, ax = plt.subplots(figsize=(8, 5))
_orden = tabla.sort_values('R2 global')
ax.barh(_orden['Modelo'], _orden['R2 global'], color=PALETTE['blue'], edgecolor='white', height=0.6)
for i, v in enumerate(_orden['R2 global']):
    ax.text(v + 0.005, i, f'{v:.3f}', va='center', fontsize=9)
ax.set_xlabel('R2 global (promedio sobre los 21 targets)')
ax.set_title('Ranking de modelos - R2 global en eval.csv', fontsize=13)
ax.set_xlim(-0.060, _orden['R2 global'].max() * 1.15)
plt.tight_layout()
plt.show()
```

Comparacion de modelos (ordenado por R2 descendente):

```
=====
          Modelo  R2 global  RMSE  MAE
1  RandomForest Multisalida  0.0964  0.9198  0.7026
2             RF Individual  0.0904  0.9178  0.6926
3      XGBoost Multisalida  0.0835  0.9187  0.6838
4  Random Forest con RegressorChain  0.0809  0.9184  0.6860
5      XGBoost Individual  0.0670  0.9300  0.7058
6  Regresión Individual  -0.0164  0.9665  0.7444
7             MLP Estandar  -0.0380  0.9597  0.7042
8      MLP Perdida Custom  -0.1316  0.9861  0.7187
=====
```

Mejor modelo: RandomForest Multisalida (R2 = 0.0964)



6.- Comparación por target

Se muestra el R2 de cada modelo para cada grupo biológico.

Esto permite identificar para que grupos funciona mejor cada enfoque y si hay grupos que ninguno de los modelos predice bien (posiblemente por falta de datos o por relaciones muy ruidosas con las variables predictoras).

```
In [6]: # Construimos una tabla con R2 por target para cada modelo
filas = []
for r in resultados:
    fila = {'Target': 'GLOBAL', 'Modelo': r['modelo'], 'R2': r['R2']}
    filas.append(fila)
    for t, r2_t in zip(TARGETS, r['r2_targets']):
        filas.append({'Target': t, 'Modelo': r['modelo'], 'R2': round(float(r2_t), 4)})
```

```

df_r2 = pd.DataFrame(filas).pivot(index='Target', columns='Modelo', values='R2')

# Ordenamos los targets: GLOBAL primero, luego por R2 medio descendente
orden = ['GLOBAL'] + [t for t in TARGETS]
df_r2 = df_r2.loc[orden]

# Ordenamos las columnas por R2 global descendente
orden_modelos = tabla['Modelo'].tolist()
df_r2 = df_r2[orden_modelos]

print('R2 por target y modelo:')
print('=' * 120)
print(df_r2.to_string())
print('=' * 120)
print('\nTargets con R2 medio < 0.2 en todos los modelos (posiblemente difíciles de predecir):')
media_por_target = df_r2.drop('GLOBAL').mean(axis=1)
dificiles = media_por_target[media_por_target < 0.2]
if len(dificiles) > 0:
    for t, v in dificiles.items():
        print(f' {t:30} R2 medio: {v:.4f}')
else:
    print(' Ninguno.')

# Grafico: R2 por modelo para los targets principales (Shannon y Richness de lombrices)
_targets_plot = ['earthworm_shannon_z', 'earthworm_richness_z']
_colors_plot = [PALETTE['green'], PALETTE['purple']] # colores fijos del proyecto: shannon / richness

_df_main = df_r2.loc[_targets_plot]
_x = np.arange(len(orden_modelos))
_width = 0.38

fig, ax = plt.subplots(figsize=(11, 5))
for i, (tgt, col) in enumerate(zip(_targets_plot, _colors_plot)):
    offset = (i - 0.5) * _width
    bars = ax.bar(_x + offset, _df_main.loc[tgt, orden_modelos], _width,
                  label=tgt, color=col, edgecolor='white')
    ax.bar_label(bars, fmt='%.2f', fontsize=7, padding=2)

ax.set_xticks(_x)
ax.set_xticklabels(orden_modelos, rotation=30, ha='right')
ax.set_ylabel('R2')
ax.set_title('R2 por modelo - earthworm_shannon_z vs earthworm_richness_z (eval.csv)', fontsize=13)
ax.axhline(0, color=PALETTE['gray'], linewidth=0.8)
ax.legend()
plt.tight_layout()
plt.show()

# Grafico: R2 medio (Shannon + Richness) por modelo
_media_principal = _df_main.loc[_targets_plot, orden_modelos].mean(axis=0).sort_values(ascending=False)

fig, ax = plt.subplots(figsize=(8, 5))
bars = ax.bar(_media_principal.index, _media_principal.values, color=PALETTE['blue'], edgecolor='white')
ax.bar_label(bars, fmt='%.3f', fontsize=8, padding=3)
ax.set_xticklabels(_media_principal.index, rotation=30, ha='right')
ax.set_ylabel('R2 medio (Shannon + Richness)')
ax.set_title('R2 medio de earthworm_shannon_z y earthworm_richness_z por modelo (eval.csv)', fontsize=13)
ax.axhline(0, color=PALETTE['gray'], linewidth=0.8)
plt.tight_layout()
plt.show()

```

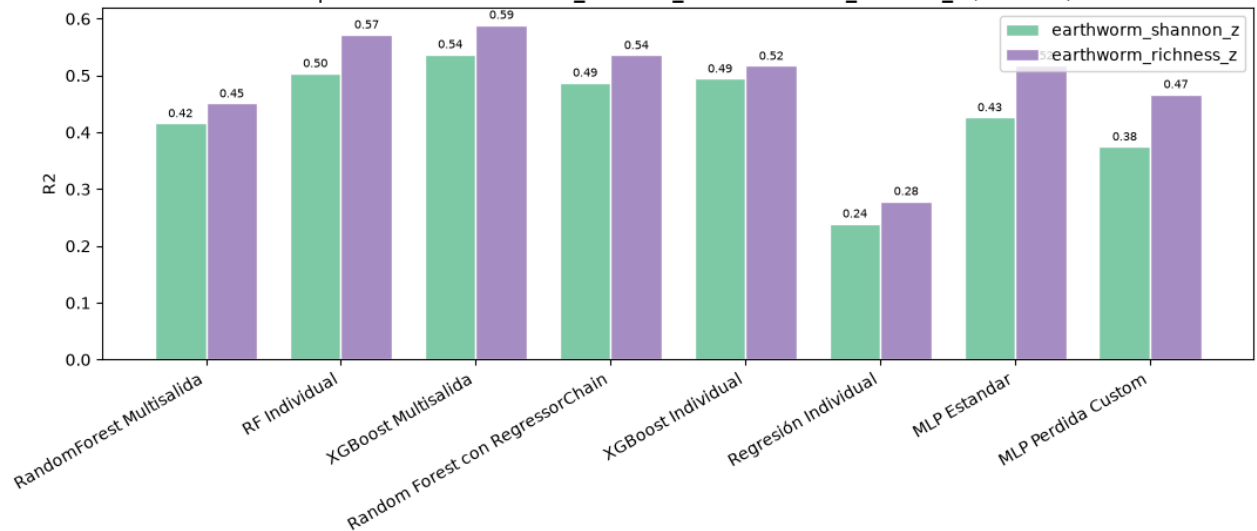
R2 por target y modelo:

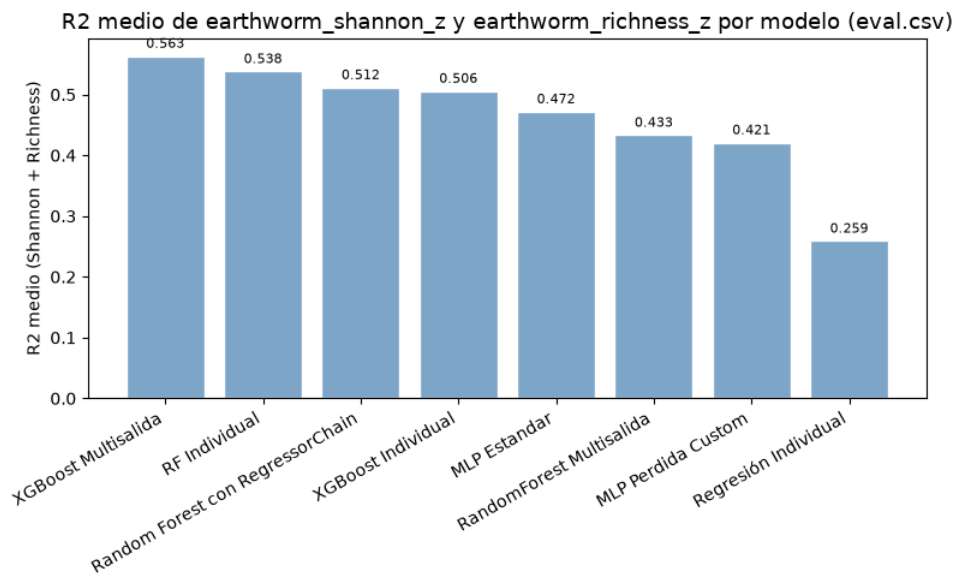
Modelo	RandomForest	Multisalida	RF Individual	XGBoost Multisalida	Random Forest con RegressorChain	XGBoost Indiv
idua	Regresión Individual	MLP Estandar	MLP Perdida Custom			
Target						
GLOBAL			0.0964	0.0904	0.0835	0.0809
0.0670	-0.0164	-0.0380	-0.1316			
nematode_shannon_z			0.1545	0.1693	0.2228	0.1467
0.1637	0.0112	0.1180	0.0903			
macro_shannon_z			0.2000	0.3199	0.3771	0.2804
0.2601	0.1257	0.2571	0.2676			
earthworm_shannon_z			0.4160	0.5041	0.5375	0.4876
0.4946	0.2395	0.4264	0.3750			
orib_shannon_z			0.1693	0.1794	0.2275	0.2069
0.1147	0.1137	0.2356	0.2216			
meso_shannon_z			-0.1178	-0.1731	-0.2861	-0.1640
0.1868	-0.2499	-0.4898	-0.6189			
coll_shannon_z			-0.1953	-0.3489	-0.3229	-0.6087
0.3312	-0.4998	-0.8227	-1.3439			
bac_shannon_z			0.0290	0.0077	-0.0151	0.0288
0.0092	-0.0812	-0.0854	-0.1004			
fun_shannon_z			-0.0127	-0.0455	-0.0502	-0.0219
0.0206	-0.0161	-0.0757	-0.0842			
euk_shannon_z			0.1378	0.1683	0.0886	0.1624
0.0912	0.0407	0.1453	0.0868			
oomy_shannon_z			0.1252	0.0832	0.0620	0.0913
0.0965	0.0857	0.1534	0.0588			
cerc_shannon_z			-0.0041	-0.0318	-0.0330	-0.0006
0.0414	0.0155	-0.1361	-0.1194			
macro_order_richness_z			0.2532	0.3417	0.4013	0.4069
0.3082	0.1521	0.3525	0.3759			
earthworm_richness_z			0.4509	0.5722	0.5893	0.5359
0.5171	0.2789	0.5176	0.4671			
orib_species_richness_z			0.2386	0.2034	0.2800	0.2026
0.1259	0.0921	0.2596	0.2542			
meso_species_richness_z			-0.0255	-0.0438	-0.1146	-0.0400
0.0592	-0.0959	-0.2753	-0.3462			
coll_species_richness_z			-0.2920	-0.5484	-0.6465	-0.5799
0.5803	-0.7341	-1.4628	-2.2411			
bac_asv_richness_z			0.0044	0.0042	0.0155	-0.0052
0.0014	-0.0761	-0.1464	-0.1680			
fun_asv_richness_z			-0.0068	-0.0256	-0.0662	-0.0186
0.0242	0.0087	0.0145	0.0027			
euk_asv_richness_z			0.1903	0.2520	0.2233	0.2690
0.1671	0.0400	0.2164	0.1902			
oomy_asv_richness_z			0.1917	0.1843	0.1650	0.2019
0.1776	0.1035	0.0113	-0.0546			
cerc_asv_richness_z			0.1171	0.1250	0.0983	0.1171
0.1227	0.1019	-0.0120	-0.0770			

Targets con R2 medio < 0.2 en todos los modelos (posiblemente difíciles de predecir):

nematode_shannon_z R2 medio: 0.1346
 orib_shannon_z R2 medio: 0.1836
 meso_shannon_z R2 medio: -0.2858
 coll_shannon_z R2 medio: -0.5592
 bac_shannon_z R2 medio: -0.0259
 fun_shannon_z R2 medio: -0.0409
 euk_shannon_z R2 medio: 0.1151
 oomy_shannon_z R2 medio: 0.0945
 cerc_shannon_z R2 medio: -0.0439
 meso_species_richness_z R2 medio: -0.1251
 coll_species_richness_z R2 medio: -0.8856
 bac_asv_richness_z R2 medio: -0.0463
 fun_asv_richness_z R2 medio: -0.0144
 euk_asv_richness_z R2 medio: 0.1935
 oomy_asv_richness_z R2 medio: 0.1226
 cerc_asv_richness_z R2 medio: 0.0741

R2 por modelo - earthworm_shannon_z vs earthworm_richness_z (eval.csv)





7.- Exportación de las comparaciones de regresión

```
In [7]: os.makedirs('output/regresion', exist_ok=True)

# Guardamos la tabla comparativa para referencia futura
tabla.to_csv('output/regresion/comparacion_modelos.csv', index=False)
df_r2.to_csv('output/regresion/r2_por_target.csv')

print('Resultados guardados en output/regresion/comparacion_modelos.csv y output/regresion/r2_por_target.csv')
```

Resultados guardados en output/regresion/comparacion_modelos.csv y output/regresion/r2_por_target.csv

8.- Prueba de clasificación

En este notebook también se realiza una pequeña prueba de clasificación para comprobar si los rankings cambian cuando se hace las pruebas teniendo en cuenta los resultados de regresión o no.

```
In [8]: # Paso 1: Carga de datos y preparación segura de percentiles
print("Iniciando Análisis Avanzado con Métricas de Clasificación Blanda...")
try:
    # Intentamos cargar train.csv usando la ruta configurada en el notebook
    df_train_tmp = pd.read_csv(os.path.join(DATA_DIR, 'train.csv'))
    df_combinado = pd.concat([df_train_tmp[TARGETS], df_eval[TARGETS]], axis=0)
    print("[INFO] Cargado train.csv con éxito. Umbrales calculados sobre train + eval.")
except Exception as e:
    df_combinado = df_eval[TARGETS]
    print("[WARN] No se pudo cargar train.csv, usando solo eval.csv para umbrales:", e)

os.makedirs('output/regresion/graphs', exist_ok=True)

y_true = y_eval.values

# Paso 2: Recalcular predicciones de todos los modelos de forma explícita y segura
dicc_preds = {}
print("\nGenerando predicciones para los 8 modelos...")

# 1. Random Forest individual
print(" [1/8] Recalculando RF Individual...")
dicc_preds['RF Individual'] = predecir_ensamble_por_target('rf', lambda t, i: f'rf_prod_{t}_m{i}.pkl', 5, X_eval)

# 2. XGBoost individual
print(" [2/8] Recalculando XGBoost Individual...")
dicc_preds['XGBoost Individual'] = predecir_ensamble_por_target('xgb', lambda t, i: f'xgb_prod_{t}_exp{i}.pkl', 5, X_eval)

# 3. Ridge individual
print(" [3/8] Recalculando Regresión Individual (Ridge)...")
y_pred_ridge = np.zeros((len(X_eval), len(TARGETS)))
for j, target in enumerate(TARGETS):
    m = joblib.load(os.path.join(MODELS_BASE, 'reg', f'ridge_prod_{target}.pkl'))
    y_pred_ridge[:, j] = m.predict(X_eval)
dicc_preds['Regresión Individual'] = y_pred_ridge

# 4. RF multisalida
print(" [4/8] Recalculando RandomForest Multisalida...")
dicc_preds['RandomForest Multisalida'] = predecir_ensamble_pkl('rf_multi', 'rf_multi_m', '', 5, X_eval)

# 5. XGBoost multisalida
print(" [5/8] Recalculando XGBoost Multisalida...")
dicc_preds['XGBoost Multisalida'] = predecir_ensamble_pkl('xgb_multi', 'xgb_multi_exp', '', 5, X_eval)

# 6. RegressorChain
print(" [6/8] Recalculando Random Forest con RegressorChain...")
N_CHAINS = 10
preds_chain = []
for chain_id in range(N_CHAINS):
    cadena = joblib.load(os.path.join(MODELS_BASE, 'chain', f'chain_{chain_id}.pkl'))
```

```

        preds_chain.append(cadena.predict(X_eval))
    dicc_preds['Random Forest con RegressorChain'] = np.mean(preds_chain, axis=0)

# 7. MLP estandar
print(" [7/8] Recalculando MLP Estandar...")
dicc_preds['MLP Estandar'] = predecir_ensamble_mlp('mlp', 'mlp_prod_', 5, X_eval, 'mlp_config.pkl')

# 8. MLP con perdida personalizada
print(" [8/8] Recalculando MLP Perdida Custom...")
dicc_preds['MLP Perdida Custom'] = predecir_ensamble_mlp('mlp_custom', 'mlp_custom_', 5, X_eval, 'mlp_custom_config.pkl')

print("\nPredicciones obtenidas con éxito!")

# Paso 3: Calcular métricas combinadas (Regresión R² + Clasificación)
filas_radar = []

for nombre_modelo, y_pred in dicc_preds.items():
    # Extraer el R2 global calculado previamente en la Celda 4 de resultados
    r2_global = 0.0
    for res in resultados:
        if res['modelo'] == nombre_modelo:
            r2_global = res['R2']
            break

    # Calcular métricas de clasificación blanda por target discretizando en Bajo/Medio/Alto
    precisions, recalls, f1s = [], [], []
    for j, target in enumerate(TARGETS):
        valores_ref = df_combinado[target].dropna().values
        p33 = np.percentile(valores_ref, 33)
        p66 = np.percentile(valores_ref, 66)

        t_disc = np.zeros_like(y_true[:, j], dtype=int)
        t_disc[y_true[:, j] > p33] = 1
        t_disc[y_true[:, j] > p66] = 2

        p_disc = np.zeros_like(y_pred[:, j], dtype=int)
        p_disc[y_pred[:, j] > p33] = 1
        p_disc[y_pred[:, j] > p66] = 2

        precisions.append(precision_score(t_disc, p_disc, average='macro', zero_division=0))
        recalls.append(recall_score(t_disc, p_disc, average='macro', zero_division=0))
        f1s.append(f1_score(t_disc, p_disc, average='macro', zero_division=0))

    precision_m = np.mean(precisions)
    recall_m = np.mean(recalls)
    f1_m = np.mean(f1s)

    # Puntuación compuesta balanceada (R2 acotado a 0 si es muy bajo/negativo para visualización uniforme)
    r2_clipped = max(0, r2_global)
    score_compuesto = (r2_clipped + precision_m + recall_m + f1_m) / 4

    filas_radar.append({
        'Modelo': nombre_modelo,
        'R2': r2_global,
        'Precision': precision_m,
        'Recall': recall_m,
        'F1_Macro': f1_m,
        'Score_Compuesto': score_compuesto
    })

df_radar_orig = pd.DataFrame(filas_radar).sort_values(by='Score_Compuesto', ascending=False)

# Paso 4: Generación del Mosaico de Gráficos de Radar
categorias = ['R²', 'Precisión', 'Recall', 'F1-Macro']
N = len(categorias)
angulos = [n / float(N) * 2 * np.pi for n in range(N)]
angulos += angulos[:1]

fig, axes = plt.subplots(2, 4, figsize=(22, 11), subplot_kw=dict(projection='polar'))
axes = axes.flatten()

os.makedirs('output', exist_ok=True)

for idx, (index, fila) in enumerate(df_radar_orig.iterrows()):
    ax = axes[idx]

    # Valores mapeados (R2 acotado a 0 para mantener escala proporcional positiva de 0 a 1)
    valores = [max(0, fila['R2']), fila['Precision'], fila['Recall'], fila['F1_Macro']]
    valores += valores[:1]

    ax.set_theta_offset(np.pi / 2)
    ax.set_theta_direction(-1)

    plt.sca(ax)
    plt.xticks(angulos[:-1], categorias, color=PALETTE['gray_dark'], size=10)
    ax.set_rlabel_position(0)
    plt.yticks([0.2, 0.4, 0.6, 0.8], ["0.2", "0.4", "0.6", "0.8"], color=PALETTE['gray'], size=8)
    plt.ylim(0, 1)

    # Dibujar poligono en el mosaico
    ax.plot(angulos, valores, linewidth=2, linestyle='solid')
    ax.fill(angulos, valores, color=PALETTE['blue'], alpha=0.15)
    ax.set_title(f"{fila['Modelo']}\nScore: {fila['Score_Compuesto']:.3f}", fontsize=11, fontweight='bold', pad=12)

# Guardar gráfico radial individual
fig_ind, ax_ind = plt.subplots(figsize=(6, 6), subplot_kw=dict(projection='polar'))
ax_ind.set_theta_offset(np.pi / 2)
ax_ind.set_theta_direction(-1)
ax_ind.plot(angulos, valores, linewidth=2, linestyle='solid', color=PALETTE['green'])
ax_ind.fill(angulos, valores, color=PALETTE['green'], alpha=0.15)

```



```

ax_ind.set_title(f"Análisis Radial: {fila['Modelo']}\nScore Compuesto: {fila['Score_Compuesto']:.4f}", fontsize=12, fontweight='b')
ax_ind.set_xticklabels(categorias)
ax_ind.set_ylim(0, 1)

nombre_archivo = f"output/regresion/graphs/radar_regression_{fila['Modelo'].lower().replace(' ', '_')}.png"
fig_ind.savefig(nombre_archivo, dpi=300, bbox_inches='tight')
plt.close(fig_ind)

# Formatear mosaico general y guardar
fig.suptitle("Análisis Comparativo Radial Individualizado (Métricas Continuas R² + Clasificación Blanda)", fontsize=16, fontweight='b')
fig.tight_layout()
fig.savefig('output/regresion/graphs/mosaico_radar_regression.png', dpi=300, bbox_inches='tight')
plt.show()

# Paso 5: Imprimir el Ranking Top 8 Final

print(" Top 8 modelos (Métricas de Regresión Original + Clasificación Blanda) ")

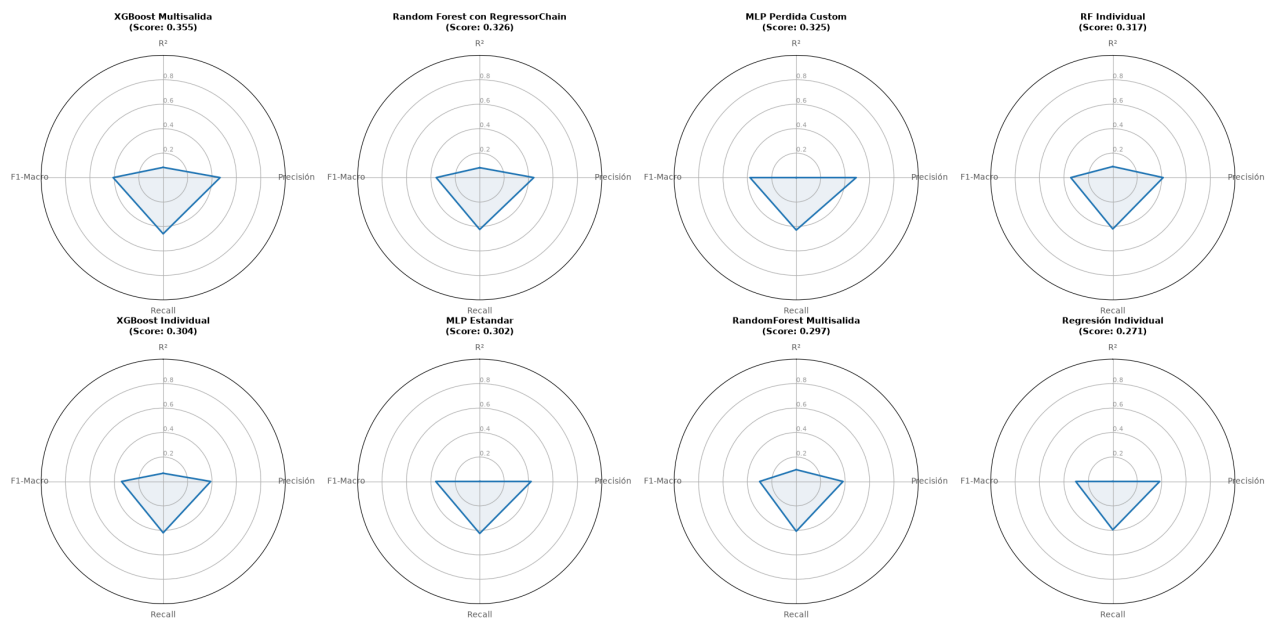
for i, (idx, fila) in enumerate(df_radar_orig.iterrows(), 1):
    print(f"Top {i}: {fila['Modelo']}<35} | Compuesto: {fila['Score_Compuesto']:.4f} | R2: {fila['R2']>7.4f} | P: {fila['Precision']

```

Iniciando Análisis Avanzado con Métricas de Clasificación Blanda...
[INFO] Cargado train.csv con éxito. Umbrales calculados sobre train + eval.

Generando predicciones para los 8 modelos...
[1/8] Recalculando RF Individual...
[2/8] Recalculando XGBoost Individual...
[3/8] Recalculando Regresión Individual (Ridge)...
[4/8] Recalculando RandomForest Multisalida...
[5/8] Recalculando XGBoost Multisalida...
[6/8] Recalculando Random Forest con RegressorChain...
[7/8] Recalculando MLP Estandar...
[8/8] Recalculando MLP Perdida Custom...
¡Predicciones obtenidas con éxito!

Análisis Comparativo Radial Individualizado (Métricas Continuas R² + Clasificación Blanda)



Top 8 modelos (Métricas de Regresión Original + Clasificación Blanda)

Top	Modelo	Compuesto	R2	P	R	F1
Top 1:	XGBoost Multisalida	0.3549	0.0835	0.4664	0.4598	0.4101
Top 2:	Random Forest con RegressorChain	0.3263	0.0809	0.4433	0.4246	0.3564
Top 3:	MLP Perdida Custom	0.3248	-0.1316	0.4911	0.4292	0.3789
Top 4:	RF Individual	0.3169	0.0904	0.4125	0.4203	0.3443
Top 5:	XGBoost Individual	0.3044	0.0670	0.3894	0.4198	0.3413
Top 6:	MLP Estandar	0.3017	-0.0380	0.4213	0.4253	0.3603
Top 7:	RandomForest Multisalida	0.2968	0.0964	0.3834	0.4073	0.3002
Top 8:	Regresión Individual	0.2706	-0.0164	0.3839	0.3952	0.3034

In [9]: # Paso 6: Metricas de clasificacion blanda (R2, Precision, Recall, F1) POR TARGET para cada modelo
print("Calculando metricas por target para el analisis por target...")

```

# Mapeo fijo modelo → color, para que cada modelo tenga siempre el mismo color
# en todos los radares por target (facilita comparar visualmente entre targets)
modelos_lista = list(dicc_preds.keys())
color_por_modelo = {m: c for m, c in zip(modelos_lista, palette_cycle(len(modelos_lista)))}

filas_radar_target = []
for nombre_modelo, y_pred in dicc_preds.items():
    r2_por_target = r2_score(y_true, y_pred, multioutput='raw_values')

    for j, target in enumerate(TARGETS):
        valores_ref = df_combinado[target].dropna().values
        p33 = np.percentile(valores_ref, 33)
        p66 = np.percentile(valores_ref, 66)

        t_disc = np.zeros_like(y_true[:, j], dtype=int)
        t_disc[y_true[:, j] > p33] = 1
        t_disc[y_true[:, j] > p66] = 2

        p_disc = np.zeros_like(y_pred[:, j], dtype=int)
        p_disc[y_pred[:, j] > p33] = 1
        p_disc[y_pred[:, j] > p66] = 2

```

```

precision_t = precision_score(t_disc, p_disc, average='macro', zero_division=0)
recall_t    = recall_score(t_disc, p_disc, average='macro', zero_division=0)
f1_t        = f1_score(t_disc, p_disc, average='macro', zero_division=0)
r2_t        = float(r2_por_target[j])
score_t     = (max(0, r2_t) + precision_t + recall_t + f1_t) / 4

filas_radar_target.append({
    'Target': target,
    'Modelo': nombre_modelo,
    'R2': r2_t,
    'Precision': precision_t,
    'Recall': recall_t,
    'F1_Macro': f1_t,
    'Score_Compuesto': score_t
})

df_radar_target = pd.DataFrame(filas_radar_target)
print(f"Metricas por target calculadas para {df_radar_target['Modelo'].nunique()} modelos "
      f"x {df_radar_target['Target'].nunique()} targets...")

```

Calculando metricas por target para el analisis por target...
Metricas por target calculadas para 8 modelos x 21 targets...

```

In [10]: # Paso 7: Radar por target - Top 8 modelos (ranking completo de los 8 modelos, por target)
os.makedirs('output/regresion/graphs/por_target', exist_ok=True)

n_targets = len(TARGETS)
n_cols_t  = 3
n_rows_t  = int(np.ceil(n_targets / n_cols_t))

fig_t, axes_t = plt.subplots(n_rows_t, n_cols_t, figsize=(6 * n_cols_t, 5.5 * n_rows_t),
                             subplot_kw=dict(projection='polar'))
axes_t = axes_t.flatten()

print(" Top 8 modelos por target (R2 + Clasificación) ")

for k, target in enumerate(TARGETS):
    ax = axes_t[k]
    df_t = (df_radar_target[df_radar_target['Target'] == target]
            .sort_values('Score_Compuesto', ascending=False)
            .reset_index(drop=True))

    ax.set_theta_offset(np.pi / 2)
    ax.set_theta_direction(-1)
    plt.sca(ax)
    plt.xticks(angulos[:-1], categorias, color=PALETTE['gray_dark'], size=8)
    ax.set_rlabel_position(0)
    plt.yticks([0.2, 0.4, 0.6, 0.8], ["0.2", "0.4", "0.6", "0.8"], color=PALETTE['gray'], size=6)
    plt.ylim(0, 1)

    for i, fila in df_t.iterrows():
        valores_t = [max(0, fila['R2']), fila['Precision'], fila['Recall'], fila['F1_Macro']]
        valores_t += valores_t[1:]
        color_m = color_por_modelo[fila['Modelo']]
        ax.plot(angulos, valores_t, linewidth=1.4, linestyle='solid',
                color=color_m, label=fila['Modelo'] if k == 0 else None)
        ax.fill(angulos, valores_t, color=color_m, alpha=0.08)

    ax.set_title(f"{target}\n(Top 1: {df_t.iloc[0]['Modelo']})", fontsize=9, fontweight='bold', pad=10)

    # Diagrama de araña individual por target
    fig_ind_t, ax_ind_t = plt.subplots(figsize=(6, 6), subplot_kw=dict(projection='polar'))
    ax_ind_t.set_theta_offset(np.pi / 2)
    ax_ind_t.set_theta_direction(-1)
    plt.sca(ax_ind_t)
    plt.xticks(angulos[:-1], categorias, color=PALETTE['gray_dark'], size=10)
    plt.yticks([0.2, 0.4, 0.6, 0.8], ["0.2", "0.4", "0.6", "0.8"], color=PALETTE['gray'], size=8)
    plt.ylim(0, 1)
    for i, fila in df_t.iterrows():
        color_m = color_por_modelo[fila['Modelo']]
        valores_t = [max(0, fila['R2']), fila['Precision'], fila['Recall'], fila['F1_Macro']]
        valores_t += valores_t[1:]
        ax_ind_t.plot(angulos, valores_t, linewidth=2, linestyle='solid', color=color_m, label=fila['Modelo'])
        ax_ind_t.fill(angulos, valores_t, color=color_m, alpha=0.08)
    ax_ind_t.set_title(f"Análisis Radial por Target: {target}", fontsize=12, fontweight='bold', pad=15)
    ax_ind_t.legend(loc='upper right', bbox_to_anchor=(1.4, 1.1), fontsize=7)
    fig_ind_t.tight_layout()
    fig_ind_t.savefig(f"output/regresion/graphs/por_target/radar_{target}.png", dpi=200, bbox_inches='tight')
    plt.close(fig_ind_t)

    # Ranking Top 8 impreso para este target
    print(f"\n{target}:")
    for i, fila in df_t.iterrows():
        print(f" Top {i+1}: {fila['Modelo']}<35 | Compuesto: {fila['Score_Compuesto']:.4f} | "
              f"R2: {fila['R2']>7.4f} | P: {fila['Precision']:.4f} | R: {fila['Recall']:.4f} | "
              f"F1: {fila['F1_Macro']:.4f}")

for j in range(n_targets, len(axes_t)):
    axes_t[j].set_visible(False)

handles, labels = axes_t[0].get_legend_handles_labels()
fig_t.legend(handles, labels, loc='lower center', ncol=4, bbox_to_anchor=(0.5, -0.01), fontsize=10)
fig_t.suptitle("Análisis Radial por Target – Ranking de los 8 modelos (R2 + Clasificación Blanda)",
               fontsize=16, fontweight='bold', y=1.01)
fig_t.tight_layout()
fig_t.savefig('output/regresion/graphs/mosaico_radar_por_target.png', dpi=250, bbox_inches='tight')
plt.show()

```

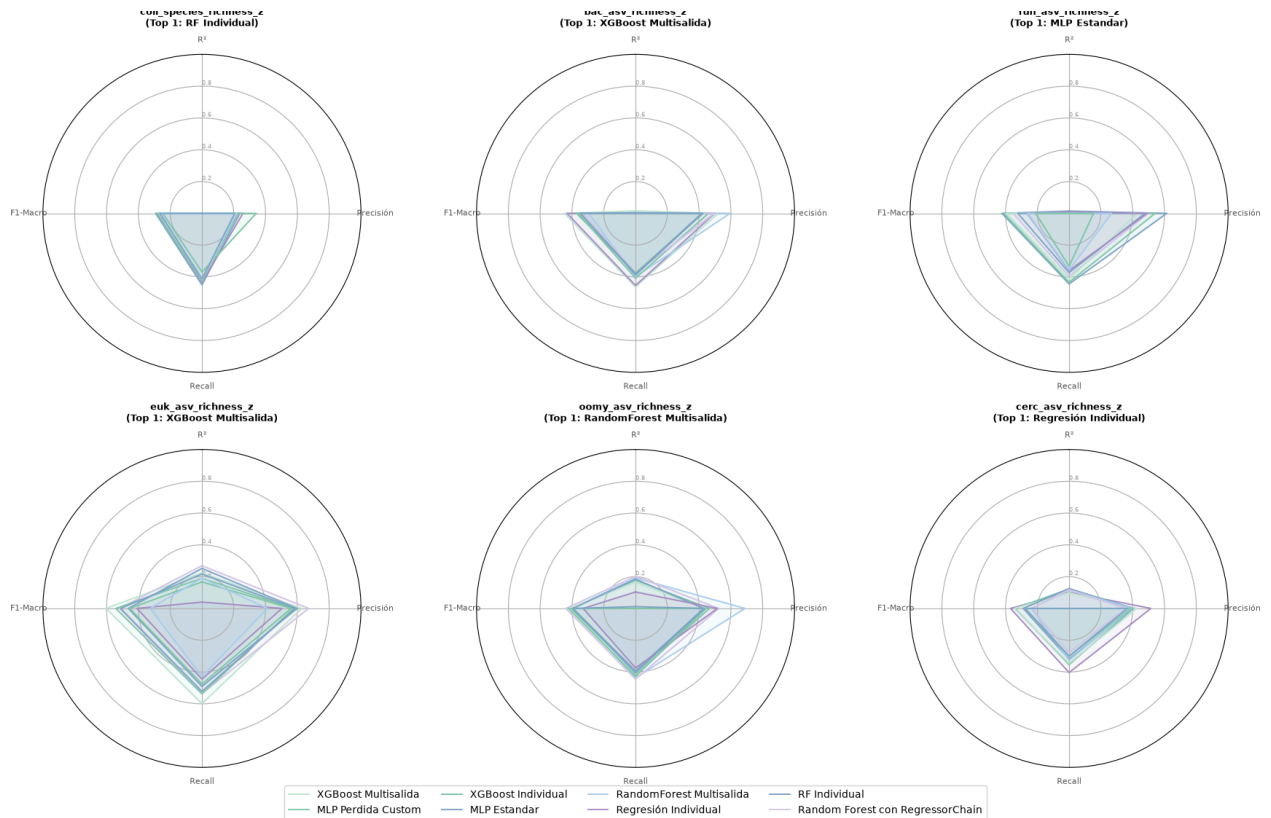
Top 8 modelos por target (R2 + Clasificación)

nematode_shannon_z:										
Top 1: XGBoost Multisalida		Compuesto: 0.4013		R2: 0.2228		P: 0.5036		R: 0.4738		F1: 0.4049
Top 2: MLP Perdida Custom		Compuesto: 0.3130		R2: 0.0903		P: 0.4591		R: 0.3883		F1: 0.3143
Top 3: XGBoost Individual		Compuesto: 0.2938		R2: 0.1637		P: 0.2889		R: 0.4065		F1: 0.3160
Top 4: MLP Estandar		Compuesto: 0.2788		R2: 0.1180		P: 0.3053		R: 0.3899		F1: 0.3022
Top 5: RandomForest Multisalida		Compuesto: 0.2742		R2: 0.1545		P: 0.2791		R: 0.3855		F1: 0.2776
Top 6: Regresión Individual		Compuesto: 0.2714		R2: 0.0112		P: 0.4122		R: 0.3615		F1: 0.3008
Top 7: RF Individual		Compuesto: 0.2694		R2: 0.1693		P: 0.2564		R: 0.3732		F1: 0.2788
Top 8: Random Forest con RegressorChain		Compuesto: 0.2638		R2: 0.1467		P: 0.2564		R: 0.3732		F1: 0.2788
macro_shannon_z:										
Top 1: XGBoost Multisalida		Compuesto: 0.5151		R2: 0.3771		P: 0.6217		R: 0.5334		F1: 0.5282
Top 2: RF Individual		Compuesto: 0.4743		R2: 0.3199		P: 0.6400		R: 0.4832		F1: 0.4541
Top 3: MLP Estandar		Compuesto: 0.4669		R2: 0.2571		P: 0.6148		R: 0.5087		F1: 0.4870
Top 4: Random Forest con RegressorChain		Compuesto: 0.4644		R2: 0.2804		P: 0.6679		R: 0.4698		F1: 0.4392
Top 5: MLP Perdida Custom		Compuesto: 0.4521		R2: 0.2676		P: 0.5754		R: 0.4848		F1: 0.4806
Top 6: XGBoost Individual		Compuesto: 0.3967		R2: 0.2601		P: 0.5141		R: 0.4295		F1: 0.3834
Top 7: RandomForest Multisalida		Compuesto: 0.3482		R2: 0.2000		P: 0.4633		R: 0.4133		F1: 0.3160
Top 8: Regresión Individual		Compuesto: 0.2953		R2: 0.1257		P: 0.3532		R: 0.3988		F1: 0.3033
earthworm_shannon_z:										
Top 1: XGBoost Multisalida		Compuesto: 0.7030		R2: 0.5375		P: 0.7703		R: 0.7578		F1: 0.7462
Top 2: RF Individual		Compuesto: 0.6711		R2: 0.5041		P: 0.7425		R: 0.7275		F1: 0.7101
Top 3: Random Forest con RegressorChain		Compuesto: 0.6678		R2: 0.4876		P: 0.7468		R: 0.7275		F1: 0.7093
Top 4: XGBoost Individual		Compuesto: 0.6429		R2: 0.4946		P: 0.7051		R: 0.6925		F1: 0.6794
Top 5: RandomForest Multisalida		Compuesto: 0.5822		R2: 0.4160		P: 0.6952		R: 0.6366		F1: 0.5808
Top 6: MLP Estandar		Compuesto: 0.5802		R2: 0.4264		P: 0.6630		R: 0.6234		F1: 0.6080
Top 7: MLP Perdida Custom		Compuesto: 0.5248		R2: 0.3750		P: 0.5887		R: 0.5768		F1: 0.5587
Top 8: Regresión Individual		Compuesto: 0.3851		R2: 0.2395		P: 0.4279		R: 0.4720		F1: 0.4008
orib_shannon_z:										
Top 1: MLP Perdida Custom		Compuesto: 0.4351		R2: 0.2216		P: 0.7284		R: 0.4717		F1: 0.3186
Top 2: MLP Estandar		Compuesto: 0.3299		R2: 0.2356		P: 0.3918		R: 0.4359		F1: 0.2564
Top 3: XGBoost Multisalida		Compuesto: 0.3094		R2: 0.2275		P: 0.3141		R: 0.4282		F1: 0.2677
Top 4: RandomForest Multisalida		Compuesto: 0.2896		R2: 0.1693		P: 0.3889		R: 0.3974		F1: 0.2028
Top 5: RF Individual		Compuesto: 0.2658		R2: 0.1794		P: 0.3286		R: 0.3641		F1: 0.1911
Top 6: Random Forest con RegressorChain		Compuesto: 0.2413		R2: 0.2069		P: 0.2385		R: 0.3231		F1: 0.1969
Top 7: XGBoost Individual		Compuesto: 0.2379		R2: 0.1147		P: 0.3046		R: 0.3718		F1: 0.1606
Top 8: Regresión Individual		Compuesto: 0.2256		R2: 0.1137		P: 0.2992		R: 0.3385		F1: 0.1512
meso_shannon_z:										
Top 1: MLP Perdida Custom		Compuesto: 0.1924		R2: -0.6189		P: 0.3597		R: 0.2296		F1: 0.1803
Top 2: RandomForest Multisalida		Compuesto: 0.1247		R2: -0.1178		P: 0.0936		R: 0.2667		F1: 0.1385
Top 3: XGBoost Individual		Compuesto: 0.1213		R2: -0.1868		P: 0.0962		R: 0.2500		F1: 0.1389
Top 4: RF Individual		Compuesto: 0.1213		R2: -0.1731		P: 0.0962		R: 0.2500		F1: 0.1389
Top 5: Regresión Individual		Compuesto: 0.1203		R2: -0.2499		P: 0.0943		R: 0.2500		F1: 0.1370
Top 6: XGBoost Multisalida		Compuesto: 0.1202		R2: -0.2861		P: 0.1037		R: 0.2333		F1: 0.1436
Top 7: Random Forest con RegressorChain		Compuesto: 0.1149		R2: -0.1640		P: 0.1143		R: 0.2000		F1: 0.1455
Top 8: MLP Estandar		Compuesto: 0.1030		R2: -0.4898		P: 0.0889		R: 0.2000		F1: 0.1231
coll_shannon_z:										
Top 1: XGBoost Multisalida		Compuesto: 0.3556		R2: -0.3229		P: 0.6290		R: 0.5306		F1: 0.2628
Top 2: MLP Perdida Custom		Compuesto: 0.3388		R2: -1.3439		P: 0.6250		R: 0.5102		F1: 0.2200
Top 3: RF Individual		Compuesto: 0.2052		R2: -0.3489		P: 0.1231		R: 0.5000		F1: 0.1975
Top 4: XGBoost Individual		Compuesto: 0.2052		R2: -0.3312		P: 0.1231		R: 0.5000		F1: 0.1975
Top 5: RandomForest Multisalida		Compuesto: 0.2052		R2: -0.1953		P: 0.1231		R: 0.5000		F1: 0.1975
Top 6: Regresión Individual		Compuesto: 0.2052		R2: -0.4998		P: 0.1231		R: 0.5000		F1: 0.1975
Top 7: Random Forest con RegressorChain		Compuesto: 0.2052		R2: -0.6087		P: 0.1231		R: 0.5000		F1: 0.1975
Top 8: MLP Estandar		Compuesto: 0.2052		R2: -0.8227		P: 0.1231		R: 0.5000		F1: 0.1975
bac_shannon_z:										
Top 1: Random Forest con RegressorChain		Compuesto: 0.3583		R2: 0.0288		P: 0.5940		R: 0.4522		F1: 0.3582
Top 2: XGBoost Individual		Compuesto: 0.3523		R2: 0.0092		P: 0.5667		R: 0.4450		F1: 0.3885
Top 3: XGBoost Multisalida		Compuesto: 0.3378		R2: -0.0151		P: 0.4828		R: 0.4451		F1: 0.4232
Top 4: Regresión Individual		Compuesto: 0.3155		R2: -0.0812		P: 0.4671		R: 0.4438		F1: 0.3511
Top 5: MLP Estandar		Compuesto: 0.3089		R2: -0.0854		P: 0.4563		R: 0.4210		F1: 0.3584
Top 6: MLP Perdida Custom		Compuesto: 0.3050		R2: -0.1004		P: 0.4511		R: 0.4126		F1: 0.3563
Top 7: RandomForest Multisalida		Compuesto: 0.2682		R2: 0.0290		P: 0.2573		R: 0.4583		F1: 0.3284
Top 8: RF Individual		Compuesto: 0.2426		R2: 0.0077		P: 0.2388		R: 0.4208		F1: 0.3030
fun_shannon_z:										
Top 1: RF Individual		Compuesto: 0.2879		R2: -0.0455		P: 0.5322		R: 0.3881		F1: 0.2313
Top 2: XGBoost Multisalida		Compuesto: 0.2779		R2: -0.0502		P: 0.4184		R: 0.3876		F1: 0.3056
Top 3: Random Forest con RegressorChain		Compuesto: 0.2653		R2: -0.0219		P: 0.4819		R: 0.3615		F1: 0.2178
Top 4: MLP Estandar		Compuesto: 0.2200		R2: -0.0757		P: 0.3145		R: 0.3373		F1: 0.2283
Top 5: MLP Perdida Custom		Compuesto: 0.2190		R2: -0.0842		P: 0.3125		R: 0.3265		F1: 0.2371
Top 6: RandomForest Multisalida		Compuesto: 0.1656		R2: -0.0127		P: 0.1864		R: 0.3270		F1: 0.1490
Top 7: XGBoost Individual		Compuesto: 0.1345		R2: -0.0206		P: 0.0781		R: 0.3333		F1: 0.1266
Top 8: Regresión Individual		Compuesto: 0.1338		R2: -0.0161		P: 0.0769		R: 0.3333		F1: 0.1250
euk_shannon_z:										
Top 1: RandomForest Multisalida		Compuesto: 0.4391		R2: 0.1378		P: 0.7989		R: 0.4444		F1: 0.3755
Top 2: RF Individual		Compuesto: 0.4391		R2: 0.1683		P: 0.6367		R: 0.4817		F1: 0.4695
Top 3: Random Forest con RegressorChain		Compuesto: 0.4273		R2: 0.1624		P: 0.6293		R: 0.4658		F1: 0.4518
Top 4: XGBoost Multisalida		Compuesto: 0.4042		R2: 0.0886		P: 0.5492		R: 0.4872		F1: 0.4917
Top 5: MLP Estandar		Compuesto: 0.3940		R2: 0.1453		P: 0.5362		R: 0.4527		F1: 0.4419
Top 6: XGBoost Individual		Compuesto: 0.3733		R2: 0.0912		P: 0.5303		R: 0.4382		F1: 0.4334
Top 7: MLP Perdida Custom		Compuesto: 0.3713		R2: 0.0868		P: 0.4927		R: 0.4555		F1: 0.4501
Top 8: Regresión Individual		Compuesto: 0.2908		R2: 0.0407		P: 0.4467		R: 0.3575		F1: 0.3182
oomy_shannon_z:										
Top 1: MLP Estandar		Compuesto: 0.4294		R2: 0.1534		P: 0.6846		R: 0.4651		F1: 0.4147
Top 2: XGBoost Multisalida		Compuesto: 0.4172		R2: 0.0620		P: 0.5660		R: 0.5250		F1: 0.5156
Top 3: MLP Perdida Custom		Compuesto: 0.3970		R2: 0.0588		P: 0.5464		R: 0.4917		F1: 0.4910
Top 4: XGBoost Individual		Compuesto: 0.3853		R2: 0.0965		P: 0.5317		R: 0.4655		F1: 0.4477
Top 5: Regresión Individual		Compuesto: 0.3625		R2: 0.0857		P: 0.6149		R: 0.3948		F1: 0.3544
Top 6: RandomForest Multisalida		Compuesto: 0.3615		R2: 0.1252		P: 0.5174		R: 0.4409		F1: 0.3626
Top 7: Random Forest con RegressorChain		Compuesto: 0.3571		R2: 0.0913		P: 0.5182		R: 0.4218		F1: 0.3969
Top 8: RF Individual		Compuesto: 0.3276		R2: 0.0832		P: 0.4772		R: 0.3893		F1: 0.3608
cerc_shannon_z:										
Top 1: XGBoost Individual		Compuesto: 0.2852		R2: -0.0414		P: 0.4397		R: 0.3633		F1: 0.3378
Top 2: Random Forest con RegressorChain		Compuesto: 0.2820		R2: -0.0006		P: 0.4247		R: 0.3698		F1: 0.3333
Top 3: XGBoost Multisalida		Compuesto: 0.2723		R2: -0.0330		P: 0.3886		R: 0.3518		F1: 0.3489
Top 4: MLP Estandar		Compuesto: 0.2666		R2: -0.1361		P: 0.3722		R: 0.3551		F1: 0.3389
Top 5: RandomForest Multisalida		Compuesto: 0.2534		R2: -0.0041		P: 0.4107		R: 0.3330		F1: 0.2697
Top 6: RF Individual		Compuesto: 0.2473		R2: -0.0318		P: 0.3718		R: 0.3296		F1: 0.2876
Top 7: MLP Perdida Custom		Compuesto: 0.2395		R2: -0.1194		P: 0.3325		R: 0.3181		F1: 0.3075
Top 8: Regresión Individual		Compuesto: 0.2282		R2: 0.0155		P: 0.3003		R: 0.3395		F1: 0.2576

macro_order_richness_z:						
Top 1: XGBoost Multisalida		Compuesto: 0.5626		R2: 0.4013		P: 0.6825 R: 0.5942 F1: 0.5722
Top 2: Random Forest con RegressorChain		Compuesto: 0.5325		R2: 0.4069		P: 0.6775 R: 0.5372 F1: 0.5085
Top 3: RF Individual		Compuesto: 0.5060		R2: 0.3417		P: 0.6667 R: 0.5271 F1: 0.4886
Top 4: MLP Perdida Custom		Compuesto: 0.4938		R2: 0.3759		P: 0.6265 R: 0.4915 F1: 0.4813
Top 5: MLP Estandar		Compuesto: 0.4667		R2: 0.3525		P: 0.5752 R: 0.4843 F1: 0.4548
Top 6: XGBoost Individual		Compuesto: 0.4342		R2: 0.3082		P: 0.6272 R: 0.4304 F1: 0.3711
Top 7: Regresión Individual		Compuesto: 0.3320		R2: 0.1521		P: 0.5701 R: 0.3484 F1: 0.2575
Top 8: RandomForest Multisalida		Compuesto: 0.3203		R2: 0.2532		P: 0.3077 R: 0.4021 F1: 0.3182
earthworm_richness_z:						
Top 1: XGBoost Multisalida		Compuesto: 0.6163		R2: 0.5893		P: 0.6592 R: 0.6126 F1: 0.6043
Top 2: XGBoost Individual		Compuesto: 0.5994		R2: 0.5171		P: 0.6515 R: 0.6168 F1: 0.6121
Top 3: MLP Estandar		Compuesto: 0.5786		R2: 0.5176		P: 0.6124 R: 0.5967 F1: 0.5879
Top 4: Random Forest con RegressorChain		Compuesto: 0.5753		R2: 0.5359		P: 0.6077 R: 0.5808 F1: 0.5767
Top 5: MLP Perdida Custom		Compuesto: 0.5693		R2: 0.4671		P: 0.6301 R: 0.5942 F1: 0.5859
Top 6: RF Individual		Compuesto: 0.5492		R2: 0.5722		P: 0.5611 R: 0.5441 F1: 0.5194
Top 7: RandomForest Multisalida		Compuesto: 0.5150		R2: 0.4509		P: 0.5516 R: 0.5349 F1: 0.5226
Top 8: Regresión Individual		Compuesto: 0.4861		R2: 0.2789		P: 0.5683 R: 0.5492 F1: 0.5479
orib_species_richness_z:						
Top 1: MLP Perdida Custom		Compuesto: 0.4427		R2: 0.2542		P: 0.6449 R: 0.4451 F1: 0.4265
Top 2: XGBoost Multisalida		Compuesto: 0.3275		R2: 0.2800		P: 0.3027 R: 0.3951 F1: 0.3323
Top 3: Random Forest con RegressorChain		Compuesto: 0.3202		R2: 0.2026		P: 0.3154 R: 0.4136 F1: 0.3492
Top 4: MLP Estandar		Compuesto: 0.3194		R2: 0.2596		P: 0.3039 R: 0.3889 F1: 0.3253
Top 5: RF Individual		Compuesto: 0.3084		R2: 0.2034		P: 0.3027 R: 0.3951 F1: 0.3323
Top 6: RandomForest Multisalida		Compuesto: 0.2954		R2: 0.2386		P: 0.2751 R: 0.3642 F1: 0.3039
Top 7: Regresión Individual		Compuesto: 0.2875		R2: 0.0921		P: 0.3205 R: 0.4012 F1: 0.3362
Top 8: XGBoost Individual		Compuesto: 0.2765		R2: 0.1259		P: 0.2889 R: 0.3765 F1: 0.3146
meso_species_richness_z:						
Top 1: MLP Perdida Custom		Compuesto: 0.2193		R2: -0.3462		P: 0.2260 R: 0.3714 F1: 0.2799
Top 2: Regresión Individual		Compuesto: 0.1995		R2: -0.0959		P: 0.2061 R: 0.3405 F1: 0.2514
Top 3: MLP Estandar		Compuesto: 0.1778		R2: -0.2753		P: 0.1813 R: 0.3048 F1: 0.2252
Top 4: XGBoost Individual		Compuesto: 0.1619		R2: -0.0592		P: 0.1603 R: 0.2833 F1: 0.2040
Top 5: Random Forest con RegressorChain		Compuesto: 0.1552		R2: -0.0400		P: 0.1538 R: 0.2714 F1: 0.1957
Top 6: RandomForest Multisalida		Compuesto: 0.1513		R2: -0.0255		P: 0.1356 R: 0.2857 F1: 0.1839
Top 7: XGBoost Multisalida		Compuesto: 0.1486		R2: -0.1146		P: 0.1474 R: 0.2595 F1: 0.1873
Top 8: RF Individual		Compuesto: 0.1300		R2: -0.0438		P: 0.1212 R: 0.2381 F1: 0.1606
coll_species_richness_z:						
Top 1: RF Individual		Compuesto: 0.2451		R2: -0.5484		P: 0.2387 R: 0.4500 F1: 0.2917
Top 2: Regresión Individual		Compuesto: 0.2430		R2: -0.7341		P: 0.2584 R: 0.4333 F1: 0.2804
Top 3: MLP Perdida Custom		Compuesto: 0.2395		R2: -2.2411		P: 0.3419 R: 0.3698 F1: 0.2464
Top 4: XGBoost Individual		Compuesto: 0.2343		R2: -0.5803		P: 0.2271 R: 0.4250 F1: 0.2850
Top 5: Random Forest con RegressorChain		Compuesto: 0.2228		R2: -0.5799		P: 0.2178 R: 0.4167 F1: 0.2569
Top 6: XGBoost Multisalida		Compuesto: 0.2226		R2: -0.6465		P: 0.2126 R: 0.4083 F1: 0.2694
Top 7: RandomForest Multisalida		Compuesto: 0.2216		R2: -0.2920		P: 0.2130 R: 0.4167 F1: 0.2567
Top 8: MLP Estandar		Compuesto: 0.2210		R2: -1.4628		P: 0.2060 R: 0.4125 F1: 0.2654
bac_asv_richness_z:						
Top 1: XGBoost Multisalida		Compuesto: 0.3611		R2: 0.0155		P: 0.5193 R: 0.4626 F1: 0.4469
Top 2: Regresión Individual		Compuesto: 0.3457		R2: -0.0761		P: 0.4982 R: 0.4538 F1: 0.4310
Top 3: RandomForest Multisalida		Compuesto: 0.3233		R2: 0.0044		P: 0.5943 R: 0.3982 F1: 0.2964
Top 4: XGBoost Individual		Compuesto: 0.3073		R2: 0.0014		P: 0.4591 R: 0.4081 F1: 0.3605
Top 5: Random Forest con RegressorChain		Compuesto: 0.3004		R2: -0.0052		P: 0.4958 R: 0.3762 F1: 0.3297
Top 6: MLP Perdida Custom		Compuesto: 0.2927		R2: -0.1680		P: 0.4241 R: 0.3756 F1: 0.3713
Top 7: RF Individual		Compuesto: 0.2907		R2: 0.0042		P: 0.4227 R: 0.3885 F1: 0.3474
Top 8: MLP Estandar		Compuesto: 0.2863		R2: -0.1464		P: 0.4172 R: 0.3815 F1: 0.3464
fun_asv_richness_z:						
Top 1: MLP Estandar		Compuesto: 0.3739		R2: 0.0145		P: 0.6141 R: 0.4444 F1: 0.4224
Top 2: MLP Perdida Custom		Compuesto: 0.3468		R2: 0.0027		P: 0.5412 R: 0.4333 F1: 0.4099
Top 3: Random Forest con RegressorChain		Compuesto: 0.3111		R2: -0.0186		P: 0.4969 R: 0.3956 F1: 0.3519
Top 4: XGBoost Multisalida		Compuesto: 0.3095		R2: -0.0662		P: 0.4295 R: 0.4133 F1: 0.3953
Top 5: RF Individual		Compuesto: 0.2934		R2: -0.0256		P: 0.4774 R: 0.3733 F1: 0.3227
Top 6: Regresión Individual		Compuesto: 0.2813		R2: 0.0087		P: 0.4921 R: 0.3600 F1: 0.2644
Top 7: RandomForest Multisalida		Compuesto: 0.2210		R2: -0.0068		P: 0.2670 R: 0.3556 F1: 0.2614
Top 8: XGBoost Individual		Compuesto: 0.1744		R2: -0.0242		P: 0.1538 R: 0.3333 F1: 0.2105
euk_asv_richness_z:						
Top 1: XGBoost Multisalida		Compuesto: 0.5132		R2: 0.2233		P: 0.6272 R: 0.5996 F1: 0.6028
Top 2: Random Forest con RegressorChain		Compuesto: 0.4984		R2: 0.2690		P: 0.6709 R: 0.5354 F1: 0.5183
Top 3: MLP Perdida Custom		Compuesto: 0.4613		R2: 0.1902		P: 0.5755 R: 0.5375 F1: 0.5420
Top 4: MLP Estandar		Compuesto: 0.4594		R2: 0.2164		P: 0.5870 R: 0.5224 F1: 0.5118
Top 5: RF Individual		Compuesto: 0.4509		R2: 0.2520		P: 0.5980 R: 0.4906 F1: 0.4628
Top 6: XGBoost Individual		Compuesto: 0.4145		R2: 0.1671		P: 0.5605 R: 0.4762 F1: 0.4542
Top 7: Regresión Individual		Compuesto: 0.3490		R2: 0.0400		P: 0.5033 R: 0.4444 F1: 0.4083
Top 8: RandomForest Multisalida		Compuesto: 0.3393		R2: 0.1903		P: 0.4122 R: 0.4286 F1: 0.3262
oomy_asv_richness_z:						
Top 1: RandomForest Multisalida		Compuesto: 0.4292		R2: 0.1917		P: 0.6885 R: 0.4368 F1: 0.3997
Top 2: Random Forest con RegressorChain		Compuesto: 0.4021		R2: 0.2019		P: 0.5231 R: 0.4451 F1: 0.4383
Top 3: XGBoost Individual		Compuesto: 0.3662		R2: 0.1776		P: 0.4650 R: 0.4166 F1: 0.4057
Top 4: XGBoost Multisalida		Compuesto: 0.3587		R2: 0.1650		P: 0.4403 R: 0.4108 F1: 0.4188
Top 5: RF Individual		Compuesto: 0.3498		R2: 0.1843		P: 0.4286 R: 0.3930 F1: 0.3935
Top 6: Regresión Individual		Compuesto: 0.3308		R2: 0.1035		P: 0.5152 R: 0.3746 F1: 0.3299
Top 7: MLP Perdida Custom		Compuesto: 0.3234		R2: -0.0546		P: 0.4438 R: 0.4281 F1: 0.4216
Top 8: MLP Estandar		Compuesto: 0.3115		R2: 0.0113		P: 0.4339 R: 0.4062 F1: 0.3944
cerc_asv_richness_z:						
Top 1: Regresión Individual		Compuesto: 0.3472		R2: 0.1019		P: 0.5148 R: 0.4037 F1: 0.3685
Top 2: XGBoost Individual		Compuesto: 0.3053		R2: 0.1227		P: 0.4050 R: 0.3535 F1: 0.3400
Top 3: XGBoost Multisalida		Compuesto: 0.3032		R2: 0.0983		P: 0.4254 R: 0.3450 F1: 0.3441
Top 4: RF Individual		Compuesto: 0.2838		R2: 0.1250		P: 0.4021 R: 0.3194 F1: 0.2886
Top 5: RandomForest Multisalida		Compuesto: 0.2684		R2: 0.1171		P: 0.3935 R: 0.3269 F1: 0.2362
Top 6: Random Forest con RegressorChain		Compuesto: 0.2468		R2: 0.1171		P: 0.3560 R: 0.2795 F1: 0.2345
Top 7: MLP Perdida Custom		Compuesto: 0.2415		R2: -0.0770		P: 0.3868 R: 0.3013 F1: 0.2777
Top 8: MLP Estandar		Compuesto: 0.2359		R2: -0.0120		P: 0.3661 R: 0.3013 F1: 0.2763

Análisis Radial por Target — Ranking de los 8 modelos (R² + Clasificación Blanda)





```
In [11]: # Paso 8: Resumen - Top 1 modelo por cada target
df_top1_target = (
    df_radar_target.sort_values('Score_Compuesto', ascending=False)
    .groupby('Target', sort=False).first()
    .reindex(TARGETS)
    .reset_index()[['Target', 'Modelo', 'Score_Compuesto', 'R2', 'Precision', 'Recall', 'F1_Macro']]
)

print(" Mejor model para cada target ")
print(df_top1_target.to_string(index=False))

conteo_top1 = df_top1_target['Modelo'].value_counts()
print("\nNumero de targets en los que cada modelo es el Top 1:")
for modelo, count in conteo_top1.items():
    print(f" {modelo:<35}: {count} / {len(TARGETS)} targets")

# Grafico resumen: modelo Top 1 por target, coloreado por modelo (mismo color que en los radares)
fig, ax = plt.subplots(figsize=(10, 9))
colores_top1 = [color_por_modelo[m] for m in df_top1_target['Modelo']]
bars = ax.barh(df_top1_target['Target'], df_top1_target['Score_Compuesto'],
               color=colores_top1, edgecolor='white')
for bar, modelo in zip(bars, df_top1_target['Modelo']):
    ax.text(bar.get_width() + 0.01, bar.get_y() + bar.get_height() / 2, modelo, va='center', fontsize=8)
ax.set_xlabel('Score compuesto del modelo Top 1')
ax.set_title('Mejor modelo (Top 1) por target', fontsize=13, fontweight='bold')
ax.set_xlim(0, max(df_top1_target['Score_Compuesto']) * 1.3)
ax.invert_yaxis()

handles = [plt.Rectangle((0, 0), 1, 1, color=color_por_modelo[m]) for m in modelos_lista]
ax.legend(handles, modelos_lista, loc='lower right', fontsize=8, title='Modelo')

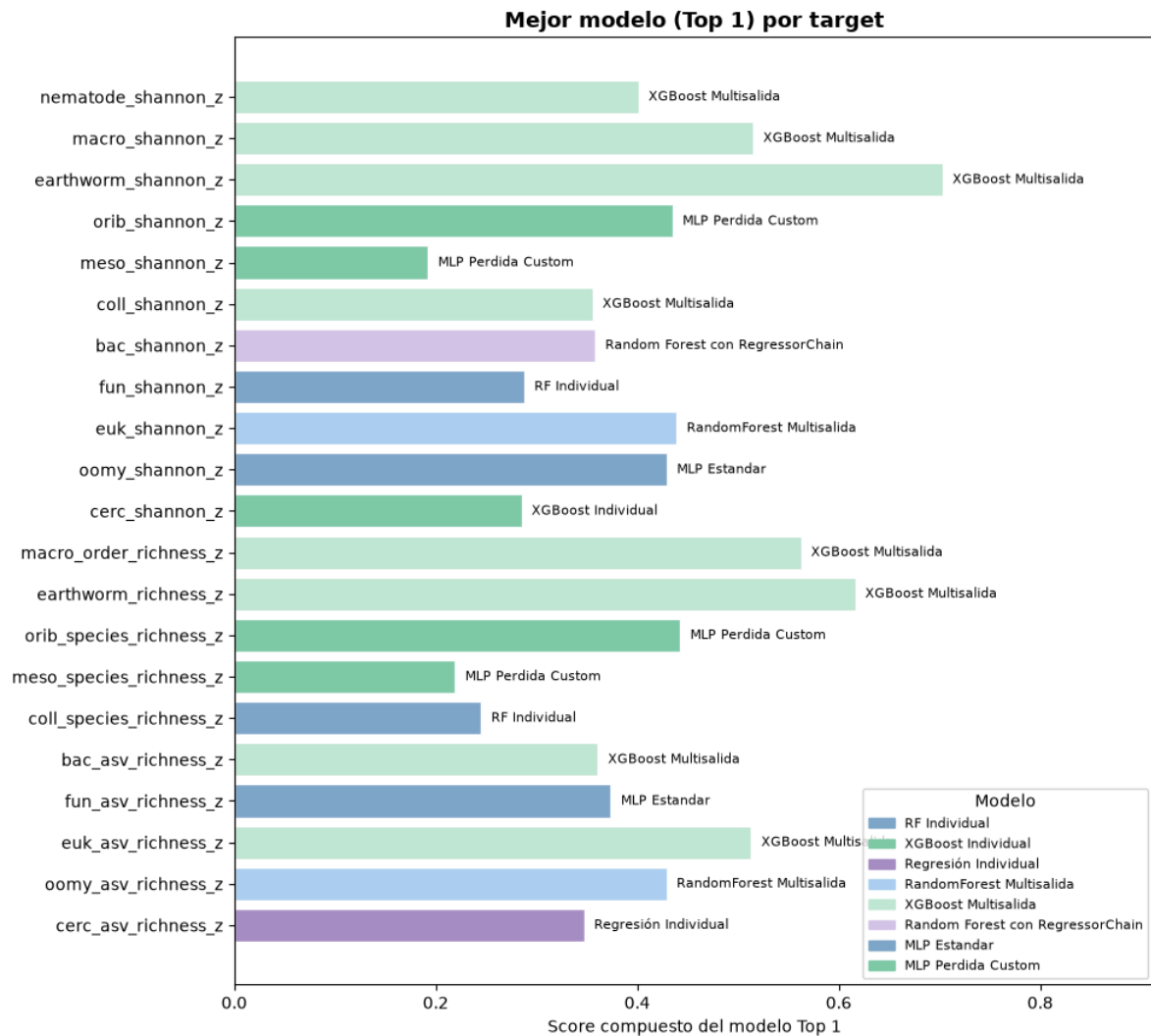
plt.tight_layout()
os.makedirs('output/regresion/graphs', exist_ok=True)
fig.savefig('output/regresion/graphs/resumen_top1_por_target.png', dpi=250, bbox_inches='tight')
plt.show()

df_top1_target.to_csv('output/regresion/top1_modelo_por_target.csv', index=False)
print("\nResumen guardado en output/regresion/top1_modelo_por_target.csv")
```

Mejor model para cada target		Modelo	Score_Compuesto	R2	Precision	Recall	F1_Macro
nematode_shannon_z		XGBoost Multisalida	0.401273	0.222791	0.503589	0.473847	0.404867
macro_shannon_z		XGBoost Multisalida	0.515109	0.377138	0.621693	0.533402	0.528205
earthworm_shannon_z		XGBoost Multisalida	0.702971	0.537528	0.770330	0.757842	0.746183
orib_shannon_z		MLP Perdida Custom	0.435079	0.221609	0.728395	0.471706	0.318606
meso_shannon_z		MLP Perdida Custom	0.192409	-0.618857	0.359690	0.229630	0.180317
coll_shannon_z		XGBoost Multisalida	0.355616	-0.322850	0.629032	0.530612	0.262821
bac_shannon_z	Random Forest con RegressorChain		0.358293	0.028837	0.594017	0.452155	0.358162
fun_shannon_z	RF Individual		0.287878	-0.045471	0.532164	0.388068	0.231282
euk_shannon_z	RandomForest Multisalida		0.439141	0.137792	0.798851	0.444444	0.375477
oomy_shannon_z	MLP Estandar		0.429433	0.153375	0.684569	0.465079	0.414709
cerc_shannon_z	XGBoost Individual		0.285203	-0.041371	0.439683	0.363294	0.337835
macro_order_richness_z	XGBoost Multisalida		0.562561	0.401261	0.682540	0.594221	0.572222
earthworm_richness_z	XGBoost Multisalida		0.616329	0.589276	0.659201	0.612565	0.604277
orib_species_richness_z	MLP Perdida Custom		0.442673	0.254237	0.644928	0.445062	0.426468
meso_species_richness_z	MLP Perdida Custom		0.219318	-0.346185	0.225957	0.371429	0.279887
coll_species_richness_z	RF Individual		0.245113	-0.548401	0.238749	0.450000	0.291701
bac_asv_richness_z	XGBoost Multisalida		0.361079	0.015548	0.519312	0.462600	0.446856
fun_asv_richness_z	MLP Estandar		0.373855	0.014462	0.614141	0.444444	0.422373
euk_asv_richness_z	XGBoost Multisalida		0.513211	0.223251	0.627193	0.599567	0.602834
oomy_asv_richness_z	RandomForest Multisalida		0.429158	0.191662	0.688462	0.436768	0.399743
cerc_asv_richness_z	Regresión Individual		0.347224	0.101861	0.514815	0.403690	0.368529

Numero de targets en los que cada modelo es el Top 1:

XGBoost Multisalida	: 8 / 21 targets
MLP Perdida Custom	: 4 / 21 targets
RF Individual	: 2 / 21 targets
RandomForest Multisalida	: 2 / 21 targets
MLP Estandar	: 2 / 21 targets
Random Forest con RegressorChain	: 1 / 21 targets
XGBoost Individual	: 1 / 21 targets
Regresión Individual	: 1 / 21 targets



Resumen guardado en output/regresion/top1_modelo_por_target.csv